

✂ Três Mosqueteiros

Jogo de tabuleiro multiplayer em tempo real, jogado remotamente entre dois jogadores através de **WebSockets**. O servidor (Node.js + TypeScript) concentra toda a regra de negócio; os clientes (HTML, CSS e JavaScript puro) apenas capturam as jogadas e exibem o estado recebido.

Desenvolvido como atividade da disciplina **Desenvolvimento Web 2**, do curso de Tecnologia em Análise e Desenvolvimento de Sistemas — IFPE.

Professor: Paulo Maurício Gonçalves Júnior **Estudante:** Luiz Eduardo dos Santos Silva

🎮 Como jogar

O jogo ocorre em um tabuleiro 5×5 com 3 peças escuras (mosqueteiros) e 22 peças claras (guardas).

Mosqueteiros (peças escuras)

- Sempre devem capturar uma peça adversária por turno
- Clique em um mosqueteiro para selecioná-lo — os guardas adjacentes serão destacados
- Clique em um guarda destacado para capturá-lo

Guardas (peças douradas)

- Sempre se movem para casas vazias adjacentes
- Clique em um guarda para selecioná-lo — as casas vazias adjacentes serão destacadas
- Clique em uma casa vazia destacada para mover o guarda

Condições de vitória

- 🏆 Guardas vencem se conseguirem forçar os 3 mosqueteiros a ficarem na mesma linha ou coluna
- 🏆 Mosqueteiros vencem se, na sua vez, não houver nenhum guarda adjacente a nenhum deles

O primeiro jogador a clicar em **"Iniciar conexão"** assume o papel de **mosqueteiro** e aguarda o segundo jogador. O segundo jogador a se conectar assume o papel de **guarda**, e a partida começa automaticamente. Apenas o jogador da vez pode realizar jogadas; ao final de cada jogada válida, ambos os jogadores são informados de quem joga em seguida, e do resultado quando a partida termina.

✂ Tecnologias

- **Servidor:** Node.js, TypeScript, WebSockets (ws)
- **Cliente:** HTML, CSS, JavaScript (ES Modules)
- Comunicação cliente ↔ servidor via mensagens **JSON**

📁 Estrutura do projeto

```
Web2-Lista2-LuizSilva/  
├── client/  
│   ├── index.html  
│   └── styles.css
```

```
├── js/
│   ├── main.js
│   ├── connection.js
│   ├── board.js
│   └── rulesPreview.js
├── server/
│   ├── src/
│   │   ├── game/
│   │   │   ├── board.ts
│   │   │   ├── rules.ts
│   │   │   └── types.ts
│   │   ├── protocol/
│   │   │   └── messages.ts
│   │   └── session/
│   │       ├── matchManager.ts
│   │       └── index.ts
│   ├── package.json
│   └── tsconfig.json
└── README.md
```



Como executar

O projeto precisa de **dois processos rodando ao mesmo tempo**: o servidor (regra de negócio + WebSocket) e o cliente (interface estática).

1. Pré-requisitos

- [Node.js](#) instalado (recomendado v18 ou superior)

2. Instalar as dependências do servidor

```
cd server
npm install
```

3. Rodar o servidor

O servidor é escrito em TypeScript. Há duas formas de executá-lo:

Opção A — executar diretamente com `tsx` (sem gerar arquivos `.js`, recomendado para desenvolvimento):

```
npx tsx src/index.ts
```

Opção B — transpilar para JavaScript com o compilador do TypeScript e depois rodar com Node puro:

```
npx tsc
node dist/index.js
```

Se a mensagem **"Servidor rodando na porta 3000"** aparecer no terminal, o servidor está pronto para receber conexões.

Mantenha este terminal aberto durante todo o jogo.

4. Servir o cliente

Em um **segundo terminal**, sirva a pasta `client/` como um servidor estático. Uma forma simples, sem precisar instalar extensões:

```
cd client
npx live-server
```

Isso abre automaticamente o jogo no navegador (por padrão em `http://127.0.0.1:8080`).

Alternativa: usar a extensão **Live Server** do VS Code, clicando com o botão direito em `client/index.html` → "Open with Live Server". Nesse caso, é importante abrir o Live Server a partir da pasta `client`, para que os caminhos de `styles.css` e `js/main.js` funcionem corretamente.

5. Jogar

Com o servidor e o cliente rodando, abra **duas abas (ou janelas) do navegador** no endereço servido pelo `live-server`, simulando os dois jogadores:

1. Na primeira aba, clique em **"Iniciar conexão"** → você será o mosqueteiro, aguardando o segundo jogador.
2. Na segunda aba, clique em **"Iniciar conexão"** → você será o guarda, e a partida começa.

Arquitetura da comunicação

Toda a regra de negócio (validação de jogadas, aplicação de movimentos, verificação de fim de jogo) está implementada no **servidor**, em `server/src/game/rules.ts`. Os clientes apenas:

- capturam a casa de origem e a casa de destino clicadas pelo jogador;
- enviam essas coordenadas ao servidor em formato JSON;
- recebem do servidor o novo estado do tabuleiro (também em JSON) e o exibem.

O cliente possui uma cópia local simplificada do cálculo de movimentos possíveis (`client/js/rulesPreview.js`), usada **apenas para destacar visualmente** as jogadas disponíveis antes do clique — a validação oficial da jogada é sempre feita pelo servidor, que pode aceitar ou recusar qualquer jogada recebida.